

Custom RAG Pipeline over an AI Course Textbook

Mid-Term Assignment — AI Developers Course — Retrieval-Augmented Generation System

Author: Kosta**Date:** May 28, 2026**Corpus:** AI Developers Course Textbook v2 (152 pp.)**Stack:** Python · Chroma · bge-large · Ollama

This report documents an end-to-end Retrieval-Augmented Generation (RAG) pipeline built over a 152-page AI course textbook. It implements the full pipeline — loading → cleaning → chunking → embedding → indexing → retrieval → generation → citation → evaluation — and exposes the required `answer(question)` interface. Three retrieval strategies (dense, BM25, hybrid) are compared in a controlled ablation, and a two-stage hybrid-plus-reranker design is used in production.

1 Corpus Description

The corpus is a single structured PDF — **AI Developers Course Textbook v2** (152 pages, 12 modules). Each concept in the book follows a strict repeating layout (Theoretical Foundation → Mathematical Intuition → Python Implementation → Engineering Best Practices), which the structured loader exploits to assign every chunk a `Module > Concept > Subsection` path and page range.

#	Module	#	Module
1	Introduction to ML Paradigms	7	Transformer Architecture
2	Linear Models & Optimization	8	LLMs & Prompt Engineering
3	Trees & Ensembles	9	Introduction to RAG
4	Neural Networks & Classification	10	Data Ingestion & Chunking
5	NLP Basics & Preprocessing	11	Vector Stores & Retrieval
6	Word Embeddings & Vectors	12	Project Implementation & Evaluation

Why this corpus suits RAG: Each section carries explicit `module / concept / subsection / page-range` metadata, enabling precise provenance and grounded citations. A base LLM knows generic ML concepts but not the exact module structure, code examples, and numerical details of *this* document — so retrieval is load-bearing for accurate, citable answers. The repetitive structure is also a useful stress test: some sections share near-identical boilerplate ("...can drastically alter the final outcome"), letting us probe how well retrieval separates truly relevant chunks from look-alikes.

2 System Architecture

The design separates **retrieval** (finding evidence) from **generation** (writing the answer) — the core RAG principle. The production entry point is the assignment-required `answer(question)` in `src/rag_system.py`, returning `{answer, sources, retrieved_chunks}`.

```
PDF → load_pdf_structured() → clean_text() → chunk (recursive) → bge-large embed → Chroma (HNSW)
→ Hybrid Retriever [Dense + BM25, k=20] → Cross-Encoder Rerank → top-5 → gemma3:4b → parse_citations() → answer()
```

Models used — three distinct models, one per stage

Stage	Model	Role	Why this choice
embed	BAAI/bge-large-en-v1.5	Encodes query + chunks into 1024-dim vectors for dense semantic search.	Top MTEB retrieval scores; runs on CPU; ships a retrieval-specific query prefix.
rerank	BAAI/bge-reranker-large	Cross-encoder that re-scores each (query, chunk) pair for precise final ranking.	Pairwise scoring is far more precise than bi-encoder vectors on the final shortlist.
LLM	gemma3:4b (Ollama)	Writes the final grounded answer with inline <code>[chunk_id]</code> citations.	Runs locally — free, private, no API; small enough for a laptop, capable enough to follow citation rules.

Code organization

File	Responsibility
src/utils.py	Three PDF loaders (full / page / structured), three chunkers (fixed / sentence / recursive), cleaning, JSONL I/O.
src/build_index.py	Loads docs, chunks them, embeds with bge-large, builds the persistent Chroma collection.
src/retrieval.py	Retriever (dense), BM25Retriever, HybridRetriever (RRF), production RerankingRetriever.
src/generation.py	Prompt building, Ollama call, citation parsing/filtering.
src/rag_system.py	The <code>answer()</code> entry point + Markdown report formatter.
eval/*.py	Auto-generates the gold set; runs the ablation; computes metrics.

3

Chunking Strategy

Load mode

structured

(default)

splits the PDF into one document per logical section using

font-size heuristics

in

load_pdf_structured()

.

PyMuPDF

(

fitz

)

exposes per-span font sizes, which the loader maps to the document hierarchy:

Font size	Interpreted as
≥ 28 pt	Module marker ("Module 1")
22-27 pt	Module title text (appended to the module heading)
17-21 pt	Section heading (Chapter Overview / Core Concept / Summary)
13.5-16 pt	Named subsection — only if it matches a known name; else body (e.g. math formulas)
≤ 11 pt	Body text
~10 pt + digits	Page number (skipped)

Each detected section is then chunked by LangChain's

RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=120)

,

and a

[Module > Concept > Subsection]

breadcrumb is prepended to every chunk's text.

Concrete example from the project

(

data/processed/chunks.jsonl

)

chunk text begins with an injected breadcrumb:

[Module 5 NLP Basics & Preprocessing > Core Concept: TF-IDF and BM25 > Theoretical Foundation]

"BM25 is a robust extension of TF-IDF used heavily in modern search engines and RAG retrieval pipelines because it saturates term frequency..."

metadata stored with it (flows through to the index and citations):

{

"module": "Module 5 NLP Basics & Preprocessing",

"concept": "Core Concept: TF-IDF and BM25",

"subsection": "Theoretical Foundation",

"page_start": 61,

"page_end": 61

}

Why structured + recursive:

Sections give natural semantic boundaries (theory / math / code / best-practices stay separate).

recursive

splitting tries paragraphs → lines → words → characters, cutting mid-sentence only as a last resort.

The 20% overlap (120/600) ensures a sentence split across two chunks reappears whole in the next one.

The breadcrumb lets the embedding model "see" structural context, so a query like "when to use BM25" can match the Module 11 Hybrid-Search section even though that section never names BM25 prominently.

Honest note on cutting:

because some Theoretical-Foundation sections exceed 600 characters, a few chunks

do

split mid-sentence (e.g.

sec2_chunk_2

ends on "...can drastically" and

sec2_chunk_3

resumes with the rest).

This is acceptable: the recursive splitter prefers clean boundaries first, the 120-char overlap restores the split sentence, and dense + BM25 retrieval work on meaning / terms rather than whole sentences.

Most short sections pass through as a single uncut chunk.

4

Embedding & Vector Index Choice

Embedding model

BAAI/bge-large-en-v1.5 — 1024-dim, L2-normalized, CPU-friendly. Queries use the model's retrieval prefix:

"Represent this sentence for searching relevant passages: " + query

Chosen over

all-MiniLM-L6-v2

(faster, 384-dim, weaker) and OpenAI

text-embedding-3-small

(paid API, not offline).

bge-large

balances retrieval quality with local CPU execution.

Vector store

Chroma

with an HNSW index and cosine distance, persisted under

data/processed/chroma/ :

client.create_collection(
name=collection,
metadata={"hnsw:space": "cosine",
"embed_model": EMBED_MODEL})

Local + persistent, HNSW built-in, supports metadata filtering, minimal boilerplate. A hosted Pinecone/Qdrant or Postgres+pgvector would be overkill for a 152-page corpus.

Score convention & reproducibility:

Chroma returns cosine

distance

; the retriever converts it to a similarity score via

score = 1.0 - distance

. Each record stores the embedding, chunk text, and full metadata dict, so the LLM can cite module-level provenance without re-fetching the source.

Chunk IDs are deterministic

(

pdf_<file>_sec<N>_chunk_<i>

)

, so deleting the Chroma dir and rerunning

build_index.py --clean

rebuilds an identical index.

5

Retrieval Method

Three retrievers share one interface —

retrieve(query, k) → [(chunk_id, text, score, metadata)] :

Retriever	Type	Strength	Weakness
Retriever (Dense)	Semantic (Chroma + bge)	Understands meaning ("car" ≈ "automobile")	Misses exact technical terms
BM25Retriever	Lexical (rank_bm25)	Exact-term matches (BM25, RLHF, sklearn)	Blind to synonyms
HybridRetriever	RRF fusion of both	Combines lexical + semantic	Slightly slower

The

production search

is a two-stage

RerankingRetriever

:

Stage 1: Hybrid (Dense + BM25, fetch k=20) → Stage 2: Cross-Encoder rerank → return top-5

Why RRF (Reciprocal Rank Fusion):

Dense cosine scores live in [0,1] while BM25 scores are unbounded, so they cannot be averaged directly.

RRF fuses by

rank

only — each chunk scores

1/(60+rank)

from each list — needing no normalization.

K=60 is the standard from Cormack et al. (2009); chunks appearing in both lists get boosted.

Why two stages:

Stage 1 is cheap and wide (high recall — get the right chunk into the top-20).

Stage 2 is expensive but precise — the cross-encoder reads query+chunk together and pushes the most relevant to rank 1.

This is the standard precision/recall split used in production retrieval.

6

Prompt Design

The system prompt

(

src/generation.py

)

enforces strict grounding rules:

Answer

only

from the provided context chunks.

If the answer is absent, reply explicitly

"I do not know"

— turning refusals into a measurable signal for negation/absence questions.

Cite chunks inline as

[chunk_id]

in square brackets.

Never invent facts, numbers, or names.

Keep answers to 2-3 focused sentences.

Context is formatted with a structural header so the model can reason about provenance without extra look-ups:

[chunk_id] (section='Module 9 > RAG > Theoretical', pages=p103-104)
<chunk text>

After generation, citations are extracted with the regex

\{\{([^\}]{s,})+_chunk_\d+\}\}

and

intersected with the retrieved IDs

— any ID the model hallucinates but never received is silently dropped, so a returned citation always points to a chunk that was actually in context.

7

Evaluation Results

A gold set of questions (one per chunk, ~182 from the structured base index) is generated by `eval/build_gold_set.py` using the same `gemma3:4b` model, cycling deterministically through five categories with a fixed seed. Negation items carry empty `must_cite_chunk_ids` by design and are scored separately. Three metrics are computed:

Metric	What it measures	Range
Hit@k	Fraction of grounded questions where a correct chunk appears in the top-k retrieved results (recall — "did it find the evidence?").	0-1
MRR@k	Mean reciprocal rank of the first correct chunk — rank 1→1.0, rank 2→0.5, rank 3→0.33 (rank quality — "how high did it rank the evidence?").	0-1
Answer Acc.	Mean token-level F1 between generated and reference answer, over the first 20 grounded items (end-to-end — "did generation use the evidence correctly?").	0-1

How Answer Accuracy (token F1) works

F1 is the harmonic mean of **precision** (of the words the model produced, how many were correct) and **recall** (of the words in the reference answer, how many the model covered). Stop-words and punctuation are stripped first, so "the / is / are" don't inflate the score. F1 rewards correct content without demanding exact wording — the right fit for short 1-3-sentence answers, where BLEU/ROUGE (built for translation/summarization) fit poorly. Hit@k is always ≥ MRR@k, since MRR additionally penalizes a low rank.

Category coverage in the gold set

The five categories — **factual**, **numerical**, **temporal**, **comparison**, **negation** — rotate deterministically across chunks. Negation/absence questions test whether the system correctly refuses ("The information was not found in the provided context.") instead of fabricating an answer — a distinct and important behavior to measure.

8

Ablation Table

Two experiments share one table. **Experiment 1** varies top-k under dense retrieval (same index, only how many chunks reach the LLM — trading recall against context noise). **Experiment 2** varies the retrieval algorithm at k=5. The run below used `RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=120)` over the structured sections, producing a `collection_size = 219`. All rows use Dense/BM25/Hybrid **without** the production reranker, to isolate each component's contribution.

Config	Retriever	k	Coll.	Hit@k	MRR@k	Ans Acc	Notes
dense_k3	dense	3	219	0.719	0.644	0.290	Focused — may miss evidence ranked 4+
dense_k5	dense	5	219	0.775	0.657	0.270	Baseline (Exp 1 mid-point, Exp 2 dense entry)
dense_k8	dense	8	219	0.838	0.666	0.303	Best Hit@k — broader recall, more noise
bm25_k5	BM25	5	219	0.750	0.630	0.310	Lexical — wins on named entities, best Ans Acc
hybrid_k5	hybrid	5	219	0.775	0.670	0.234	RRF fusion — best MRR@k

Answer accuracy = mean token-level F1 between generated and reference answer, over the first 20 grounded gold-set items. Two cells are highlighted: **dense_k8** (best recall) and **hybrid_k5** (best rank quality).

Reading the results: raising k from 3→8 in dense retrieval lifts Hit@k by ~12 points (0.719→0.838) as more candidates enter the window, while MRR moves only ~2 points (0.644→0.666) — recall improves but the correct chunk, when found, is already near the top. **Hybrid wins on MRR@k (0.670)**, the best rank quality of any config, because RRF rewards chunks that both retrievers agree on. **BM25 wins on answer accuracy (0.310)**, since exact-term questions (module names, library names) are answered most faithfully from lexically-matched chunks. The standout low value is hybrid's answer accuracy (0.234): fusing two pools sometimes surfaces a semantically-plausible-but-wrong chunk at rank 1, which the small generator then paraphrases incorrectly — exactly the case a cross-encoder reranker is meant to fix.

Why chunk-size was NOT used as an ablation axis: in `structured` mode every section is already short, so the splitter's effect is limited; the meaningful axes that actually move the metrics on this corpus are *retriever type* and *top-k*, so the ablation focuses there rather than on a parameter with little effect.

9

Failure Analysis

A real two-run example on the query "*when need to use BM25?*" demonstrates the dominant failure mode and how the design recovers from it:

Run	Outcome	Cause
"BM57" (typo)	Partial — explained <i>what</i> BM25 is, not <i>when</i> to use it. The decisive Module 11 Hybrid-Search chunk (<code>sec160</code>) did not reach top-5.	Typo broke BM25 lexical matching, so the use-case chunk was missed.
"BM257" (correct)	Full — combined Module 5 (what BM25 is) + Module 11 ("Dense is great for semantics but bad for exact keyword matching"). A genuine multi-hop answer.	Correct query let Hybrid + reranker surface the right evidence from two modules.

Other errors observed in the manually classified sample (`eval/answers_sample.jsonl`):

- Retrieval-noise false refusals:** the gold chunk was present in top-5 but ranked below a cluster of look-alike boilerplate chunks, so the LLM saw no clear answer and refused. The production cross-encoder reranker over top-20 directly targets this.
- Citation hallucination:** on a code-snippet question the model cited a chunk ID not in the retrieved set; the citation filter silently dropped it, so the final answer carried no invented source.
- Answer-accuracy plateau:** despite Hit@8 of 0.838, token-F1 stays ~0.23-0.31 — `gemma3:4b` paraphrases rather than quoting exact numbers and subsection names, costing partial-match F1, and this is worst for the fused hybrid pool.
- Boilerplate confusion:** the textbook repeats "...can drastically alter the final outcome" across dozens of sections; near-identical chunks compete in the top-k and occasionally crowd out the truly relevant one.

10

What I Would Improve Next

- Cross-encoder reranker measured on the gold set:** the production reranker is currently validated manually and from literature; quantifying its Hit@k/MRR uplift on all gold questions is the top next experiment — it should directly lift hybrid's low answer accuracy (0.234) by fixing rank-1 precision.
- Query expansion / HyDE** (Hypothetical Document Embeddings): generate a hypothetical answer and embed *that* as the query — would have surfaced the Hybrid-Search chunk in the BM25 typo example, and helps numerical/comparison questions where query phrasing diverges from the document.
- Metadata-filtered retrieval:** when a question names "Module X" or a specific subsection, pre-filter the Chroma query on the `module` field before dense search, eliminating cross-module false positives and the boilerplate-confusion failure.
- Larger generator** (`gemma3:12b` or `llama3:8b-instruct`): retrieval already reaches 0.838 Hit@8, so generation is now the active bottleneck — a stronger, instruction-tuned model would quote exact values and close the answer-accuracy gap.
- LLM-as-judge evaluation** across the full gold set, replacing the 20-item token-F1 sample with a scalable semantic-correctness signal that doesn't penalize valid paraphrases.
- De-duplication in retrieval:** collapse near-identical overlapping chunks before they reach the top-k, freeing context-window slots for genuinely distinct evidence.